"PHARMA SEARCH"

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

KEERTHANA	**:2520030168**
CHINMAYI	**:2520030144**
LIKHITHA	**:2520030580**

Team Number: 18

Section: 9

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"PHARMA SEARCH"** is a record of Bonafide work of **Keerthana – 2520030168, Chinmayi – 2520030144, Likhitha – 2520030580** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Keerthana – 2520030168

Chinmayi – 2520030144

Likhitha – 2520030580

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"PHARMA SEARCH"** is a bonafide work done and submitted by **KEERTHANA – 2520030168, CHINMAYI – 2520030144, LIKHITHA – 2520030580** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement and timely suggestions throughout the development of our project “Pharma Search”. His clear explanations of the course Data Structures and Algorithms-3 helped us to understand and apply the string matching, dynamic programming and network flow algorithms used in this work.

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and the supportive academic environment that made this project possible. We also thank all the faculty members of the department for their help and encouragement.

Finally, we thank our parents and friends for their support and motivation, and the management of Koneru Lakshmaiah Education Foundation for giving us the opportunity to carry out this project.

Keerthana – 2520030168

Chinmayi – 2520030144

Likhitha – 2520030580

Abstract

Pharma Search is a Java-based pharmaceutical information search application designed to efficiently search medicine names and pharmaceutical keywords from a predefined text corpus. The project implements the Knuth–Morris–Pratt (KMP) String Matching Algorithm to identify occurrences of a user-entered medicine name or keyword within the pharmaceutical corpus.

The system uses a pharmaceutical corpus file containing information related to medicines, their uses, antibiotics, dosage-related information, side effects, precautions, and other pharmaceutical terms. When the user enters a search query, the application builds the Longest Prefix Suffix (LPS) array for the given pattern and applies the KMP algorithm to search the corpus. The algorithm avoids unnecessary character comparisons by using previously matched information, making the searching process efficient.

The application displays the entered search query, the total number of matches found, the positions of the matching occurrences, and the sentences containing the searched term. If no match is found, the system informs the user and suggests alternative pharmaceutical keywords. The application also supports repeated searches until the user enters the exit command.

The project demonstrates the practical application of string matching algorithms, file handling, arrays, loops, methods, and user input processing in Java. The implementation provides an efficient approach for searching pharmaceutical information from a text corpus and demonstrates how the KMP algorithm can be integrated into a real-world medicine search application.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview	1-6
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives	
		1.5 Importance of String Matching	
		1.6 Development Methodology	
		1.7 Applications of the Project	
		1.8 Abbreviations and Terms Used	
2	System Requirements	2.1 Functional Requirements	7-11
		2.2 Non-Functional Requirements	
		2.3 Hardware and Software Specifications	
		2.4 Related Work and Research Gap	
		2.5 Feasibility Study	
		2.6 Use Cases	
		2.7 Assumptions and Constraints	
3	System Design	3.1 High-level Architecture Diagram	12-20
		3.2 Description of each layer	
		3.3 Algorithms and Techniques Used	
		3.4 Dataset (Pharmaceutical Corpus)	
		3.5 Data Flow Diagrams	
		3.6 Interaction between the Modules	
		3.7 Flowcharts of the Remaining Algorithms	
		3.8 Design Alternatives Considered	
		3.9 Data Structures Used	
4	Implementation	4.1 Module-wise implementation	21-36
		4.2 Menu-driven interface and input handling	
		4.3 Corpus file handling	
		4.4 Complexity analysis	
		4.5 Worked Examples (Dry Runs)	
		4.6 Method-level Description	
		4.7 Error Handling and Input Validation	
		4.8 Complete Source Code	
		4.9 Steps to Compile and Run	
5	Results	5.1 Experimental Setup	37-44
		5.2 KMP Search Results	
		5.3 Edit Distance Results	
		5.4 Network Flow Results	
		5.5 Test Cases and Analysis	
		5.6 Additional KMP Search Tests	
		5.7 Additional Spelling Check Tests	
		5.8 Performance Comparison	

		5.9 Discussion against the Objectives 5.10 Scalability and Memory	
6	Challenges & Limitations	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System 6.4 Safety and Ethical Considerations 6.5 Lessons Learnt from Testing	45-47
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development 7.4 Roadmap of Future Enhancements 7.5 Concluding Remarks	48-50
	References	- Journal papers, books and documentation sites used	51

List of Figures

1.1	Overall Block Diagram of Pharma Search	1
3.1	Layered Architecture of Pharma Search	12
3.2	Flowchart of the KMP Search Process	14
3.3	Medicine Supply Network Graph	15
3.4	Data Flow Diagram – Level 0	16
3.5	Data Flow Diagram – Level 1	16
3.6	Sequence of a KMP Search Request	17
3.7	Flowchart of the Edit Distance Method	18
3.8	Flowchart of the Maximum Flow Method	19
5.1	Main Menu of Pharma Search	37
5.2	KMP Search – Match Found	37
5.3	KMP Search – No Match Found	38
5.4	Spelling Check using Edit Distance	38
5.5	Network Flow Calculation Output	39
5.6	KMP versus Naive Search Time	42

1. INTRODUCTION

1.1 Overview

Pharmaceutical information such as the names of medicines, their uses, dosage-related information, side effects and precautions is stored in large volumes of text, for example drug leaflets, prescription notes and health articles. Finding a particular medicine or keyword in such text quickly and accurately is important for students, pharmacists and patients alike. Searching manually is slow and error-prone, and a naive character-by-character search wastes effort because it re-examines characters that have already been compared [6].

Pharma Search is a Java console application that tackles this problem using algorithms studied in the course Data Structures and Algorithms-3. At its centre is the Knuth–Morris–Pratt (KMP) string matching algorithm [1], which searches a pharmaceutical text corpus for a medicine name or keyword typed by the user without ever moving backwards in the text. Two supporting modules complete the application. An Edit Distance module [2], [3] detects a probable spelling mistake in a medicine name by comparing it with a list of known medicines, and a Network Flow module [4], [5] models the movement of medicines from a supplier through a pharmacy to a patient and computes the maximum possible flow using the Ford–Fulkerson method. Fig. 1.1 shows the overall block diagram of the system.

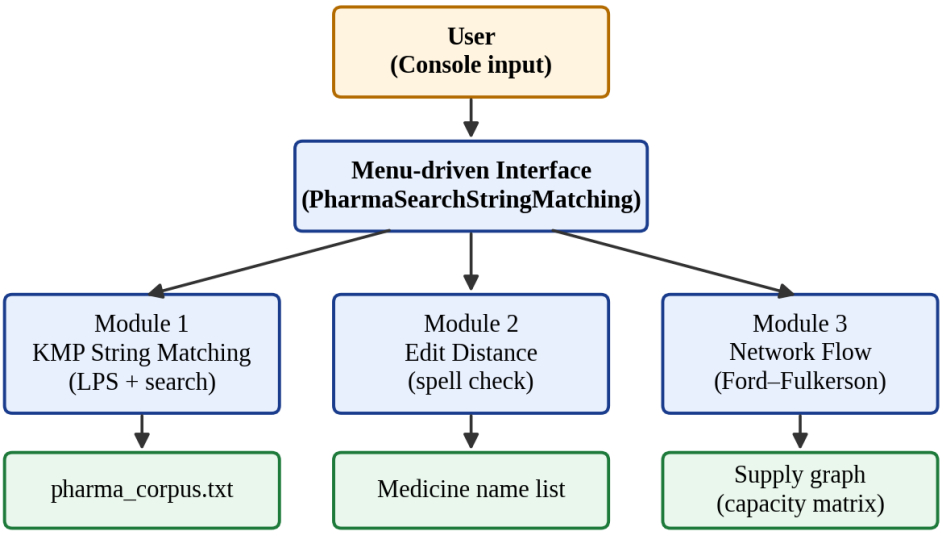


Fig. 1.1 Overall Block Diagram of Pharma Search

The user works through a simple menu that is displayed repeatedly until the user chooses to exit. At start-up the application reads the pharmaceutical corpus from the file data/pharma_corpus.txt, and all three modules are reached from the same menu. The application is written entirely in core Java and uses only arrays, strings, loops, methods, collections and file handling, which makes the working of each algorithm easy to follow.

1.2 Problem Statement

Text-based pharmaceutical information is large and unstructured, and a user who needs a specific medicine or keyword has to read through the text manually. Three practical difficulties arise. First, a simple search that restarts comparison from the beginning of the pattern after every mismatch is inefficient for long texts. Second, medicine names are long and unfamiliar, so spelling mistakes such as “paracetmol” are common, and a search for a misspelt name returns nothing, which can mislead the user into thinking that the medicine is not present. Third, medicines reach patients through a supply chain whose links have limited capacities, and it is useful to know the largest quantity that can actually be delivered.

The problem addressed by this project is therefore to design and implement an efficient Java application that (i) searches a pharmaceutical corpus for a user-entered medicine name or keyword using the KMP algorithm and displays the sentences in which it occurs, (ii) checks the spelling of a medicine name using edit distance, and (iii) computes the maximum medicine flow through a supplier–pharmacy–patient network.

1.3 Scope of the Project

The project covers a menu-driven console application that works on a predefined plain-text pharmaceutical corpus. Searching is case-insensitive and finds every occurrence of the query as a substring of the corpus. The spelling checker compares the input with a fixed list of thirteen medicine names and keywords, and the network flow module demonstrates the algorithm on a fixed five-node supply network.

The project does not include a graphical interface, a database, live pharmacy inventory or any diagnosis or prescription feature. The information in the corpus is

meant only to demonstrate the algorithms and must not be treated as medical advice. The main challenges handled during development were avoiding redundant character comparisons, comparing characters without regard to case, dealing with empty or unmatched queries, loading the corpus safely from a file, and allowing the user to perform repeated operations in one run.

The rest of this report is organised as follows. Chapter 2 lists the functional and non-functional requirements together with the hardware and software specifications and reviews related work. Chapter 3 presents the architecture and the algorithms used. Chapter 4 explains the implementation using pseudocode. Chapter 5 presents the results of testing. Chapter 6 discusses the challenges and limitations, and Chapter 7 concludes the report and suggests future work.

1.4 Objectives

The objectives of the project are as follows.

- To implement the Knuth–Morris–Pratt algorithm, including construction of the Longest Prefix Suffix (LPS) array, for searching a medicine name or keyword in a pharmaceutical corpus.
- To report the total number of matches and to display every sentence of the corpus that contains the searched term.
- To implement the dynamic programming solution of the edit distance problem and use it to detect possible spelling mistakes in medicine names.
- To model a medicine supply chain as a flow network and compute its maximum flow with the Ford–Fulkerson method.
- To provide a simple, menu-driven interface that supports repeated operations until the user exits.
- To demonstrate the practical use of string matching, dynamic programming, graph traversal, file handling, arrays, loops and methods in Java.

1.5 Importance of String Matching in the Pharmaceutical Domain

String matching is one of the oldest and most widely used operations in computing. Every time a person searches for a word in a document, a record in a file or a medicine in a list, a string matching routine is at work. In the pharmaceutical domain the operation has special value because the vocabulary is large, the names

are long and look alike (for example, “azithromycin” and “erythromycin”), and the cost of a wrong or missed result can be high. A reliable search tool helps a student to revise the uses of a medicine, helps a pharmacist to verify a name quickly, and helps a patient to find the precautions that are printed in a leaflet.

Three properties make a string matching method suitable for such work. The first is correctness: the method must report every occurrence and must never report a false one. The second is speed: the response should not depend on the pattern in a way that can make some queries unexpectedly slow. The third is tolerance: users make typing errors, and a good tool should help them to recover from these errors instead of simply saying that nothing was found. The three modules of Pharma Search correspond to these properties. The KMP module provides correct and predictable search, the edit distance module provides tolerance to spelling mistakes, and the network flow module extends the pharmaceutical theme from information to the supply of medicines.

Because the project was carried out as part of the course Data Structures and Algorithms-3, an additional aim was educational. Each algorithm is implemented in a short, self-contained Java method, and the program prints enough information for a reader to compare the output with a manual trace of the algorithm. This makes the application useful as a teaching aid as well as a small search tool.

1.6 Development Methodology

The project was developed in incremental steps. Each step produced a working program that could be tested before the next feature was added. This approach was chosen because the modules are independent of each other, so a defect found in one step could be corrected without disturbing the others. Table 1.1 lists the steps.

Table 1.1 Development phases of the project

Phase	Activity	Outcome
1	Requirement study	Features of the search tool decided; algorithms chosen from the course syllabus
2	Corpus preparation	Plain-text pharmaceutical corpus and list of medicine names created
3	KMP module	buildLPS and KMPSearch written and tested on small patterns

Phase	Activity	Outcome
4	Sentence display	Corpus split into sentences; matching sentences printed with numbering
5	Edit distance module	editDistance and checkSpelling written and tested on misspelt names
6	Network flow module	Capacity matrix, BFS and maxFlow written; result checked by hand
7	Menu and integration	main method with the repeated menu, input handling and error messages
8	Testing and documentation	Test cases executed, performance measured and this report prepared

Version control was not required for a project of this size, but the source file was backed up after every phase so that an earlier working version could be restored if a change introduced an error.

1.7 Applications of the Project

Although Pharma Search is a small console program, the techniques it uses appear in many real systems, and the project can be extended in several practical directions.

- **Study aid:** students of pharmacy and computer science can use the program to search a set of notes and to observe how string matching, approximate matching and flow computation work on real words.
- **Reference lookup:** a pharmacy or a clinic can load its own text of leaflets and use the search to find all lines that mention a medicine or a precaution.
- **Data entry checking:** the spelling check can be used when medicine names are typed into a stock register, to detect names that differ by a single letter from a known name.
- **Supply planning:** the network flow module shows how the capacity of each link limits the quantity that reaches the patient, which is the basis of simple planning of medicine distribution.

1.8 Abbreviations and Terms Used

Table 1.2 explains the technical terms and abbreviations that are used in this report.

Table 1.2 Abbreviations and terms

Term	Meaning
KMP	Knuth–Morris–Pratt string matching algorithm
LPS	Longest Prefix Suffix; the array computed from the pattern before the KMP search
Corpus	The body of pharmaceutical text in which the search is performed
Pattern	The medicine name or keyword typed by the user
Edit distance	Minimum number of insertions, deletions and substitutions that convert one string into another
Flow network	Directed graph whose edges have capacities and through which a quantity flows from a source to a sink
Residual graph	Graph of the remaining capacities after some flow has been pushed
Augmenting path	A path from source to sink along which more flow can still be pushed
BFS	Breadth-first search
JDK / JRE	Java Development Kit / Java Runtime Environment

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

The functional requirements describe what the system must do. They are summarised in Table 2.1.

Table 2.1 Functional requirements of Pharma Search

ID	Requirement	Module
FR1	Load the pharmaceutical corpus from the file data/pharma_corpus.txt when the program starts and stop with a clear message if the file cannot be loaded.	Corpus
FR2	Display a menu with four options (KMP search, spelling check, network flow, exit) and show it again after every operation until the user chooses to exit.	Menu
FR3	Accept a medicine name or keyword, build its LPS array and search the corpus with the KMP algorithm without regard to letter case.	KMP
FR4	Display the query, the total number of matches and the status MATCH FOUND or NO MATCH FOUND.	KMP
FR5	Display every sentence of the corpus that contains the query as a numbered list, or a message if no sentence matches.	KMP
FR6	Compare an entered medicine name with the stored medicine names using edit distance and display the closest name, the distance and a status (correct spelling, possible spelling mistake, or no close medicine found).	Edit Distance
FR7	Compute and display the maximum medicine flow from source to sink in the supply network.	Network Flow
FR8	Report an invalid menu choice and allow the user to try again.	Menu
FR9	Terminate gracefully with a closing message when the user selects Exit.	Menu

2.2 Non-Functional Requirements

The non-functional requirements describe the qualities expected from the system.

- **Performance:** the KMP search runs in time proportional to the length of the corpus plus the length of the query, so that the response is immediate for corpora of ordinary size.
- **Usability:** the numbered menu, the prompts and the clearly labelled result blocks make the application easy to use without any training.

- **Reliability:** a missing corpus file, an empty query and an invalid menu choice are handled without the program terminating abnormally.
- **Scalability:** the corpus is read line by line and the search time grows linearly with its size, so a larger corpus can be used without changing the code.
- **Portability:** the program is written in standard Java and runs on any operating system that has a Java Runtime Environment.
- **Maintainability:** each algorithm is implemented as a separate static method, so a module can be modified or tested independently.
- **Accuracy:** matching is exact (letter case is ignored), so no false match is reported and no genuine occurrence is missed.

2.3 Hardware and Software Specifications

Pharma Search has modest resource needs. The specifications used for development and testing are given in Table 2.2.

Table 2.2 Hardware and software specifications

Item	Specification
Processor	Intel Core i3 or equivalent (or higher)
Memory (RAM)	4 GB or more
Storage	100 MB of free disk space
Operating system	Windows 10/11, Linux or macOS
Programming language	Java (JDK 8 or later)
Development environment	Any Java IDE (for example IntelliJ IDEA, Eclipse or VS Code) or the command line
Input data	Plain-text corpus file data/pharma_corpus.txt

2.4 Related Work and Research Gap

String matching has been studied extensively. The naive method compares the pattern with every position of the text and needs O(nm) time in the worst case. Knuth, Morris and Pratt [1] showed that the pattern itself contains enough information to decide how far to shift after a mismatch, which reduces the search to O(n + m) time. Later algorithms such as Boyer–Moore [9] and Rabin–Karp [10] follow different strategies, comparing from the right end of the pattern or using hash values, and are widely used in practice. Textbooks such as [6] and [7] present all of these methods together with their analysis.

For approximate matching, Levenshtein [2] defined the distance between two strings as the minimum number of insertions, deletions and substitutions required to convert one into the other, and Wagner and Fischer [3] gave the dynamic programming algorithm that computes it. This idea is the basis of many spelling correction tools. For flow problems, Ford and Fulkerson [4] introduced the augmenting path method for the maximum flow problem, and Edmonds and Karp [5] showed that choosing the shortest augmenting path with breadth-first search gives a polynomial running time.

These algorithms are usually studied and implemented in isolation, using abstract strings and graphs. Large drug-information platforms offer keyword search, but they rely on indexing infrastructure and do not expose the underlying algorithms, so they are of little use as learning material. The gap addressed by this project is a small, self-contained application in which exact string matching, approximate matching and network flow are brought together in one pharmaceutical setting, so that the working of each algorithm can be observed on realistic data.

2.5 Feasibility Study

Before the design was started, the feasibility of the project was examined from four points of view.

- **Technical feasibility:** the project needs only the standard Java library. All three algorithms use arrays, strings and the collections ArrayList and LinkedList, which are available in every JDK. The team already had the required knowledge of Java from earlier courses, so no new technology had to be learnt.
- **Economic feasibility:** no paid software, license or hardware is required. The JDK and the editors used for development are free, and the program runs on an ordinary computer, so the development and running cost is negligible.
- **Operational feasibility:** the interface is a numbered menu that anyone who can use a keyboard can operate. No installation beyond a Java Runtime Environment is necessary, and the corpus can be replaced by editing a text file.

- **Schedule feasibility:** because the modules are independent and small, each could be completed and tested within the time available for the course project.

2.6 Use Cases

The system has a single actor, the user, who may be a student, a pharmacist or any person who wants to look up pharmaceutical information. Table 2.3 describes the use cases supported by the application.

Table 2.3 Use cases of Pharma Search

ID	Use case	Precondition	Main flow
UC1	Search a medicine or keyword	Corpus loaded successfully	User selects 1 and enters a query; the system reports the count, the status and the matching sentences
UC2	Check the spelling of a medicine	Program running	User selects 2 and enters a name; the system reports the closest stored name, the distance and a status
UC3	Find maximum medicine flow	Program running	User selects 3; the system computes and displays the maximum flow of the supply network
UC4	Exit the application	Program running	User selects 4; the system prints a closing message and stops

In the alternate flow of UC1, if the query does not occur in the corpus, the system displays the status NO MATCH FOUND and returns to the menu. In UC2, if the entered name is far from every stored name, the system reports that no close medicine was found. In every use case an invalid menu choice produces an error message and the menu is displayed again.

2.7 Assumptions and Constraints

The design of Pharma Search rests on the following assumptions and constraints.

- The corpus is a UTF-8 or ASCII plain-text file placed in the folder data under the name pharma_corpus.txt, and it is small enough to be held in memory as one string.
- Sentences in the corpus end with a full stop, a question mark or an exclamation mark followed by white space.

- The list of medicine names used for the spelling check is stored in the program and has to be edited in the source code to add a new name.
- The supply network used by the network flow module is fixed in the source code.
- One user works with the program at a time, from a console, and no network or database access is available.
- The pharmaceutical text is only a sample used to demonstrate the algorithms and is not a clinical reference.

3. SYSTEM DESIGN

3.1 High-level Architecture Diagram

Pharma Search follows a layered design in which every layer has one responsibility and talks only to the layer next to it. Fig. 3.1 shows the four layers: the presentation layer that communicates with the user, the application layer that controls the program flow, the algorithm layer that contains the three algorithms, and the data layer that supplies the corpus, the medicine names and the supply network.

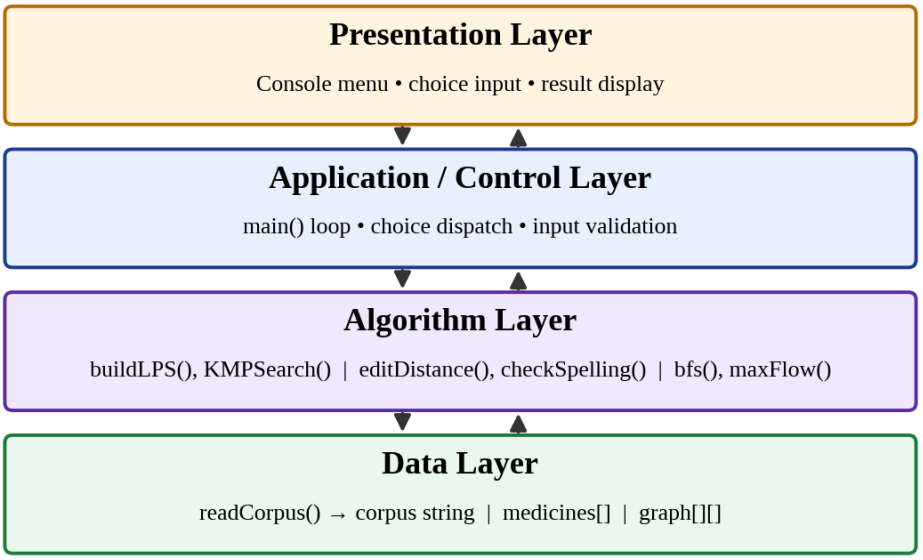


Fig. 3.1 Layered Architecture of Pharma Search

3.2 Description of each layer

3.2.1 Presentation Layer

This layer is the console. It prints the menu, reads the user's choice and query with a Scanner object, and prints the result blocks headed KMP SEARCH, EDIT DISTANCE and NETWORK FLOW. It contains no algorithmic logic.

3.2.2 Application / Control Layer

The main method forms the control layer. It loads the corpus, defines the list of medicine names and then runs an infinite loop in which it displays the menu, reads the choice and calls the appropriate module. The loop ends only when the user selects option 4, and an unrecognised choice results in the message "Invalid choice."

3.2.3 Algorithm Layer

This layer contains the core of the project. The methods buildLPS and KMPSearch implement string matching, editDistance and checkSpelling implement the spelling check, and bfs and maxFlow with networkFlow implement the maximum flow computation. The helper method displaySentences uses KMPSearch to list the matching sentences.

3.2.4 Data Layer

The data layer holds the pharmaceutical corpus, which is read from a text file by readCorpus into a single string, the array of thirteen medicine names and keywords used by the spelling checker, and the two-dimensional capacity matrix that represents the supply network.

3.3 Algorithms and Techniques Used

3.3.1 Knuth–Morris–Pratt (KMP) String Matching

KMP avoids re-comparing characters that are already known to match. Before searching, it preprocesses the pattern into the LPS array, in which lps[i] is the length of the longest proper prefix of the pattern that is also a suffix of its first i + 1 characters. When a mismatch occurs after j characters have matched, the pattern index is moved to lps[j − 1] instead of 0, and the text index never moves backwards [1]. As an illustration, Table 3.1 shows the LPS array for the pattern ABABCABAB.

Table 3.1 LPS array for the illustrative pattern ABABCABAB

Index	0	1	2	3	4	5	6	7	8
Pattern	A	B	A	B	C	A	B	A	B
LPS	0	0	1	2	0	1	2	3	4

KMP is suitable for this project because the corpus can be long, the number of searches is repeated, and the algorithm guarantees O(n + m) time irrespective of the pattern. The flowchart of the search process is given in Fig. 3.2.

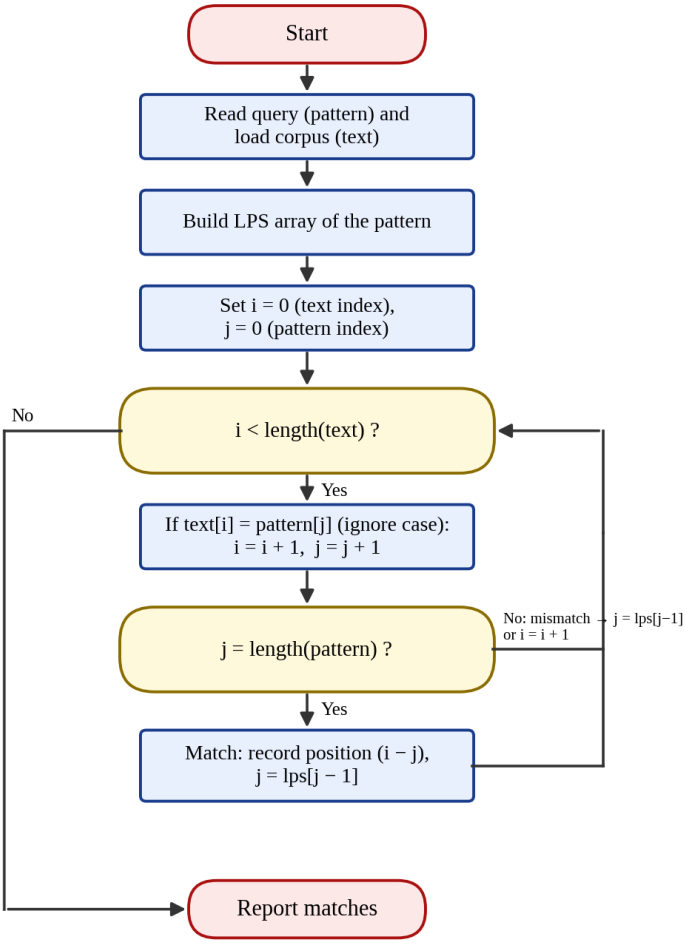


Fig. 3.2 Flowchart of the KMP Search Process

3.3.2 Edit Distance

The edit distance between a string a of length m and a string b of length n is computed with a table dp of size (m + 1) × (n + 1), where dp[i][j] is the distance between the first i characters of a and the first j characters of b [2], [3]. The first row and column are initialised to the lengths of the prefixes, because converting to or from an empty string needs that many insertions or deletions. For the remaining cells, dp[i][j] = dp[i − 1][j − 1] if the characters a[i − 1] and b[j − 1] are equal, and dp[i][j] = 1 + min(dp[i − 1][j], dp[i][j − 1], dp[i − 1][j − 1]) otherwise, which correspond to deletion, insertion and substitution. The answer is dp[m][n]. For example, “paracetmol” becomes “paracetamol” by inserting one letter, so the distance is 1. Dynamic programming is suitable because the problem has overlapping sub-problems and an optimal substructure.

3.3.3 Network Flow (Ford–Fulkerson Method)

The supply chain is modelled as a directed graph with five nodes: Source (0), Supplier (1), Pharmacy (2), Patient (3) and Sink (4). The weight of each edge is the maximum number of units that can pass through it, as shown in Fig. 3.3. The Ford–Fulkerson method [4] repeatedly finds a path from the source to the sink along edges that still have spare capacity, pushes the bottleneck amount along it, and updates a residual graph in which forward capacities decrease and reverse capacities increase. When no such path remains, the total pushed is the maximum flow. The implementation uses breadth-first search to find the paths, which is the Edmonds–Karp variant [5] and guarantees a polynomial number of augmentations.

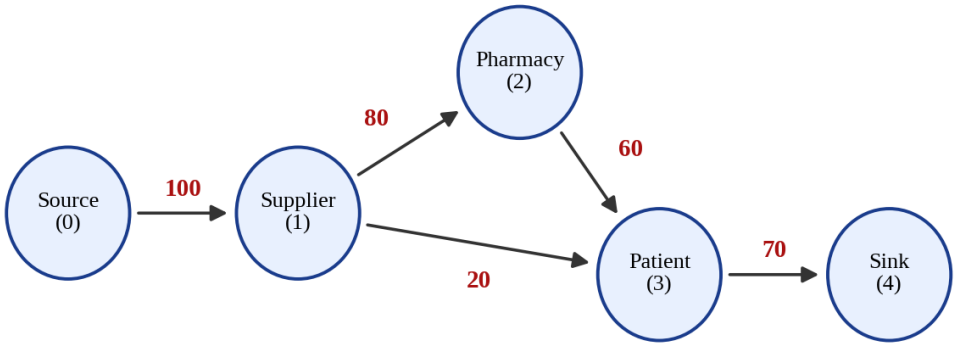


Fig. 3.3 Medicine Supply Network Graph

3.4 Dataset (Pharmaceutical Corpus)

The dataset is a plain-text file, data/pharma_corpus.txt, containing sentences about medicines such as paracetamol, ibuprofen, amoxicillin, azithromycin, cetirizine, metformin, omeprazole, aspirin, diclofenac, atorvastatin and insulin. The sentences describe the uses of the medicines, antibiotics, dosage-related information, side effects and precautions. Sentences end with a full stop, question mark or exclamation mark, which is the property used to split the text into sentences while displaying results. The file is read line by line with a BufferedReader and stored in one string so that KMP can scan it in a single pass.

3.5 Data Flow Diagrams

The flow of data through the system is shown with data flow diagrams. The level 0 diagram of Fig. 3.4 treats the whole application as one process. The user

supplies a menu choice and a query, and receives matches, sentences, a spelling status or a flow value. The corpus file and the built-in data (the medicine names and the capacity matrix) are the only data sources.

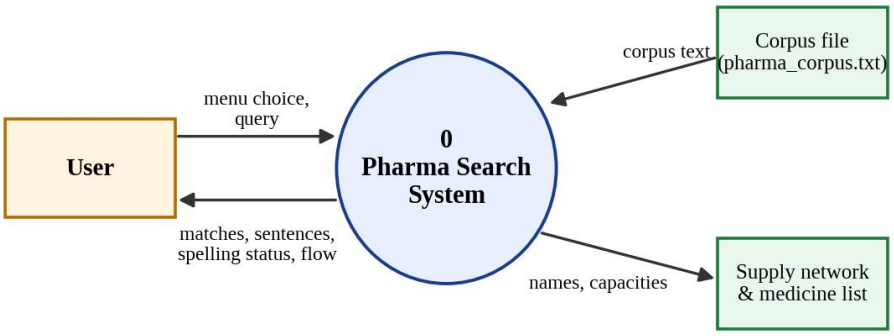


Fig. 3.4 Data Flow Diagram – Level 0

The level 1 diagram of Fig. 3.5 splits the process into four sub-processes. Process 1.0 reads the corpus file line by line and produces the corpus string held in data store D1. Process 2.0 takes the query and the corpus string and produces the match result. Process 3.0 reads the stored names from D2 and the entered name, and produces the spelling result. Process 4.0 reads the capacity matrix from D3 and produces the maximum flow. The user receives the output of every process through the same console.

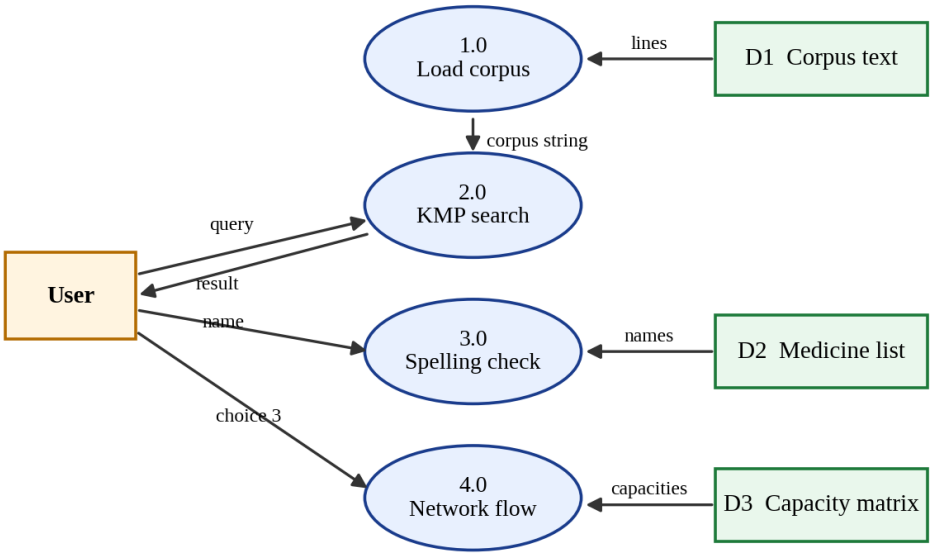


Fig. 3.5 Data Flow Diagram – Level 1

3.6 Interaction between the Modules

Fig. 3.6 shows how the methods collaborate when the user performs a KMP search. The menu code calls KMPSearch with the corpus and the query, and KMPSearch calls buildLPS to prepare the table for the pattern. After the count is known, the menu code calls displaySentences, which in turn calls KMPSearch once for every sentence. The result is returned to the user as the count, the status and the numbered sentences. The calls to the spelling check and to the network flow module follow the same pattern but are shorter, because each of them is a single method call from the menu code.

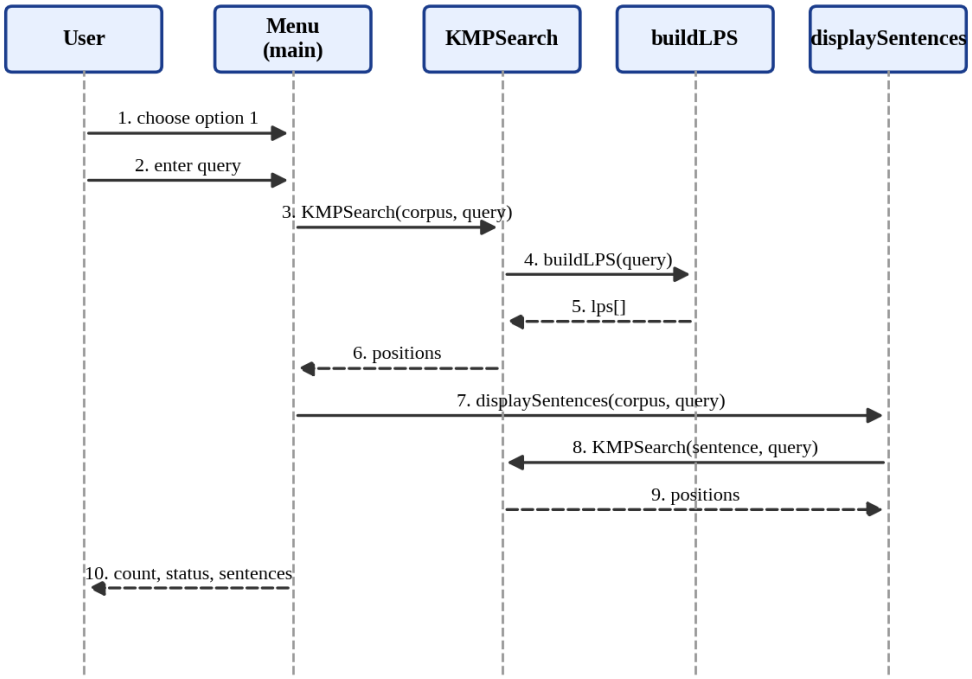


Fig. 3.6 Sequence of a KMP Search Request

3.7 Flowcharts of the Remaining Algorithms

The flowchart of the KMP search was given in Fig. 3.2. The flowcharts of the edit distance method and the maximum flow method are given in Fig. 3.7 and Fig. 3.8. In both of them, the rectangles are processing steps, the rounded boxes with a yellow background are decisions or loop tests, and the red rounded boxes mark the start and the end.

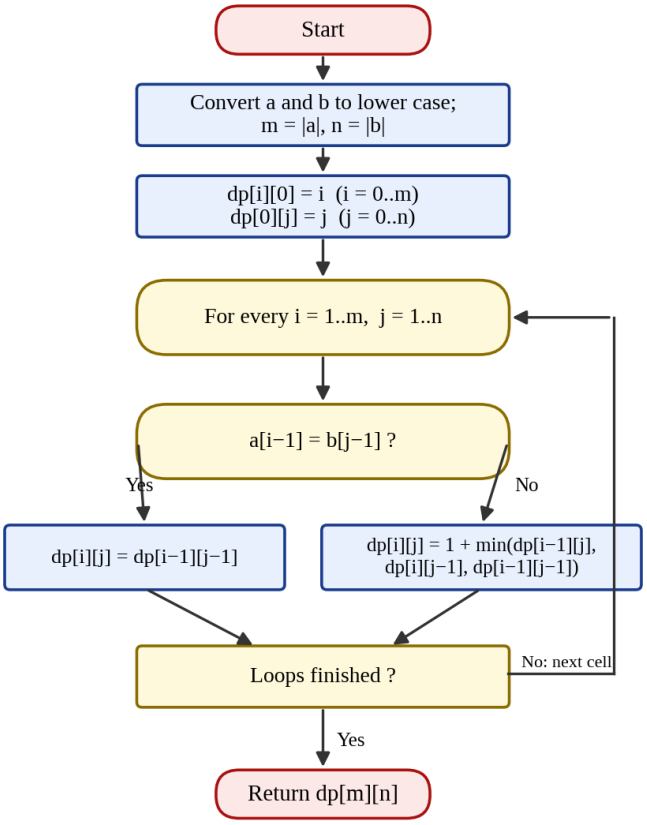


Fig. 3.7 Flowchart of the Edit Distance Method

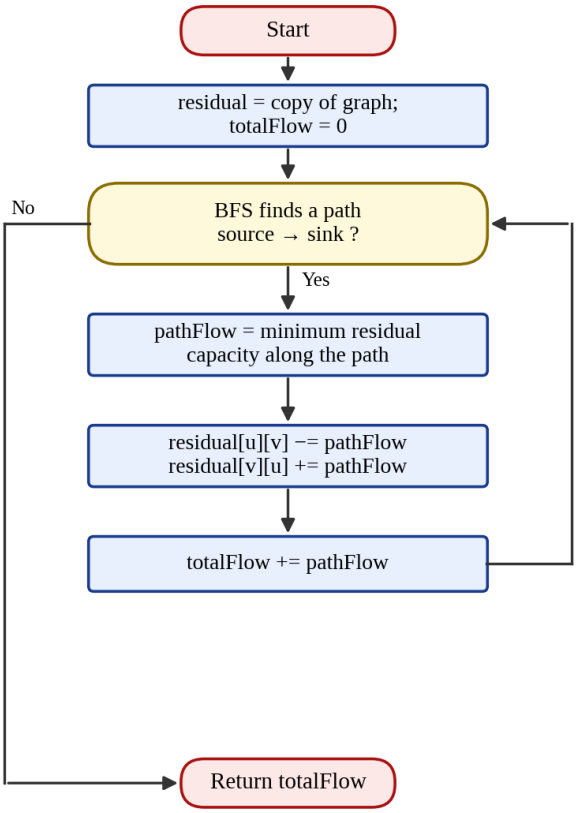


Fig. 3.8 Flowchart of the Maximum Flow Method

3.8 Design Alternatives Considered

Several alternatives were considered for each module before the final design was chosen. Table 3.2 compares the string matching methods that were studied.

Table 3.2 Comparison of string matching methods

Method	Worst-case time	Preprocessing	Remarks
Naive	O(n·m)	None	Simplest to write, but re-compares text characters after every mismatch
Rabin–Karp [10]	O(n·m) (average O(n + m))	Hash of the pattern	Good for several patterns, but a hash collision needs a character check
Boyer–Moore [9]	O(n·m) (often sub-linear)	Bad-character and good-suffix tables	Very fast in practice, but more tables and a longer implementation
Knuth–Morris–Pratt [1]	O(n + m)	LPS array of size m	Guaranteed linear time and a short implementation; chosen for this project

KMP was chosen because its linear worst-case bound does not depend on the pattern, because its single LPS table is easy to explain and to trace by hand, and

because it is part of the syllabus of the course. For the spelling check, a simple comparison of the first letters or a count of common letters was rejected because it cannot distinguish an insertion from a substitution, whereas edit distance models both exactly. For the flow computation, a depth-first search for augmenting paths was rejected because its number of augmentations depends on the capacities; the breadth-first search used in the Edmonds–Karp variant gives a bound that does not [5].

3.9 Data Structures Used

The data structures used in the program are summarised in Table 3.3. All of them come from the standard Java library.

Table 3.3 Data structures used in Pharma Search

Data structure	Used in	Purpose
String	corpus, query, sentences	Holds the text and the search pattern
StringBuilder	readCorpus	Builds the corpus efficiently while the file is read
int[] (lps)	buildLPS, KMPSearch	Stores the longest prefix-suffix length for each pattern position
ArrayList<Integer>	KMPSearch	Collects the starting positions of the matches
int[][] (dp)	editDistance	Table of distances between prefixes of the two strings
String[]	main	Stores the thirteen medicine names and the sentences after splitting
int[][] (graph, residual)	maxFlow, networkFlow	Capacity matrix and residual capacity matrix of the supply network
Queue<Integer> (LinkedList)	bfs	Holds the nodes waiting to be visited by the breadth-first search
boolean[] and int[]	bfs	Visited flags and the parent of each node on the discovered path

4. IMPLEMENTATION

The application is implemented in one Java class, PharmaSearchStringMatching, whose static methods correspond to the modules described in Chapter 3. This chapter explains each module with pseudocode. In the pseudocode, indices of strings and arrays start from 0.

4.1 Module-wise implementation

4.1.1 Module 1: KMP String Matching

The procedure BUILD-LPS constructs the LPS array of the pattern, and KMP-SEARCH uses it to find all occurrences. All character comparisons are made after converting both characters to lower case, which makes the search case-insensitive without creating a modified copy of the text.

```
BUILD-LPS(pattern)
    m ← length(pattern);  lps[0..m−1] ← 0
    len ← 0;  i ← 1
    WHILE i < m DO
        IF lower(pattern[i]) = lower(pattern[len]) THEN
            len ← len + 1;  lps[i] ← len;  i ← i + 1
        ELSE IF len ≠ 0 THEN
            len ← lps[len − 1]
        ELSE
            lps[i] ← 0;  i ← i + 1
    RETURN lps
```

```
KMP-SEARCH(text, pattern)
    positions ← empty list
    IF pattern is null OR empty THEN RETURN positions
    lps ← BUILD-LPS(pattern);  i ← 0;  j ← 0
    WHILE i < length(text) DO
        IF lower(text[i]) = lower(pattern[j]) THEN  i ← i + 1;  j ← j + 1
        IF j = length(pattern) THEN
            add (i − j) to positions
            j ← lps[j − 1]
        ELSE IF i < length(text) AND lower(text[i]) ≠ lower(pattern[j]) THEN
            IF j ≠ 0 THEN j ← lps[j − 1]
            ELSE i ← i + 1
    RETURN positions
```

The method returns the starting position of every match, including overlapping ones, and the menu reports the number of positions found. To show the

context of the matches, the procedure DISPLAY-SENTENCES splits the corpus into sentences at the characters “.”, “!” and “?” followed by white space, and prints every sentence in which KMP-SEARCH finds the query.

```
DISPLAY-SENTENCES(corpus, query)
    sentences ← split corpus after every ".", "!" or "?" followed by white space
    count ← 0
    FOR EACH sentence IN sentences DO
        IF KMP-SEARCH(sentence, query) is not empty THEN
            count ← count + 1;  PRINT count, ". ", trim(sentence)
    IF count = 0 THEN PRINT "No matching sentence found."
```

4.1.2 Module 2: Edit Distance and Spelling Check

EDIT-DISTANCE fills the dynamic programming table described in Section 3.3.2 after converting both strings to lower case. CHECK-SPELLING applies it to every stored medicine name, keeps the name with the smallest distance, and classifies the result. A distance of 0 means that the spelling is correct, a distance of 1 to 3 means that there is a possible spelling mistake, and a larger distance means that no close medicine was found.

```
EDIT-DISTANCE(a, b)
    a ← lower(a);  b ← lower(b);  m ← length(a);  n ← length(b)
    FOR i ← 0 TO m DO dp[i][0] ← i
    FOR j ← 0 TO n DO dp[0][j] ← j
    FOR i ← 1 TO m DO
        FOR j ← 1 TO n DO
            IF a[i−1] = b[j−1] THEN dp[i][j] ← dp[i−1][j−1]
            ELSE dp[i][j] ← 1 + MIN(dp[i−1][j], dp[i][j−1], dp[i−1][j−1])
    RETURN dp[m][n]
```

```
CHECK-SPELLING(query, medicines)
    closest ← "";  minDistance ← ∞
    FOR EACH medicine IN medicines DO
        d ← EDIT-DISTANCE(query, medicine)
        IF d < minDistance THEN  minDistance ← d;  closest ← medicine
    PRINT query, closest, minDistance
    IF minDistance = 0 THEN PRINT "Correct spelling"
    ELSE IF minDistance ≤ 3 THEN PRINT "Possible spelling mistake"
    ELSE PRINT "No close medicine found"
```

4.1.3 Module 3: Network Flow

The graph is stored as an adjacency (capacity) matrix. BFS searches for a path from the source to the sink in the residual graph and records each node's parent, and MAX-FLOW uses the parents to find the bottleneck, update the residual capacities and accumulate the flow.

```
BFS(graph, source, sink, parent)
    visited[·] ← false;  enqueue source;  visited[source] ← true;  parent[source] ← −1
    WHILE queue is not empty DO
        u ← dequeue
        FOR v ← 0 TO n − 1 DO
            IF NOT visited[v] AND graph[u][v] > 0 THEN
                enqueue v;  parent[v] ← u;  visited[v] ← true
    RETURN visited[sink]
```

```
MAX-FLOW(graph, source, sink)
    residual ← copy of graph;  totalFlow ← 0
    WHILE BFS(residual, source, sink, parent) DO
        pathFlow ← ∞
        FOR v ← sink; v ≠ source; v ← parent[v] DO
            pathFlow ← MIN(pathFlow, residual[parent[v]][v])
        FOR v ← sink; v ≠ source; v ← parent[v] DO
            residual[parent[v]][v] ← residual[parent[v]][v] − pathFlow
            residual[v][parent[v]] ← residual[v][parent[v]] + pathFlow
        totalFlow ← totalFlow + pathFlow
    RETURN totalFlow
```

The method networkFlow supplies the capacity matrix of Fig. 3.3, calls MAX-FLOW with node 0 as the source and node 4 as the sink, and prints the result.

4.2 Menu-driven interface and input handling

The application has no graphical front end. All user interaction takes place through the console, and the program state consists only of the loaded corpus and the array of medicine names. The main method loads the corpus, then repeatedly shows the menu and dispatches on the choice entered, as shown in the pseudocode below. Reading the choice and the query with nextLine() ensures that queries containing spaces, such as "side effects", are read completely.

```
MAIN
    corpus ← READ-CORPUS("data/pharma_corpus.txt")
    IF corpus is empty THEN PRINT error message;  STOP
    medicines ← [paracetamol, amoxicillin, ibuprofen, azithromycin, cetirizine,
        metformin,
            omeprazole, aspirin, diclofenac, atorvastatin, insulin, antibiotic, antacid]
    LOOP forever
        PRINT menu;  READ choice
        IF choice = "1" THEN
            READ query;  positions ← KMP-SEARCH(corpus, query)
            PRINT query and number of positions
            IF positions is not empty THEN PRINT "MATCH FOUND";  DISPLAY-
                SENTENCES(corpus, query)
            ELSE PRINT "NO MATCH FOUND"
        ELSE IF choice = "2" THEN  READ query;  CHECK-SPELLING(query,
            medicines)
        ELSE IF choice = "3" THEN  NETWORK-FLOW()
        ELSE IF choice = "4" THEN  PRINT closing message;  EXIT LOOP
        ELSE PRINT "Invalid choice."
```

4.3 Corpus file handling

The corpus is loaded by the method readCorpus, which opens the file through a BufferedReader, appends each line followed by a newline character to a StringBuilder, and closes the reader. If an IOException occurs, the method prints “Error reading corpus file.” and returns whatever has been read, which is an empty string when the file does not exist. The main method tests for an empty corpus and ends the program with the message “Corpus file could not be loaded.” so that no search is attempted on missing data.

```
READ-CORPUS(filePath)
    corpus ← empty StringBuilder
    TRY
        open filePath for reading
        WHILE a line can be read DO append line and newline to corpus
        close file
    CATCH IOException
        PRINT "Error reading corpus file."
    RETURN corpus as string
```

4.4 Complexity analysis

The time and space requirements of the main procedures are given in Table 4.1, where n is the length of the text, m the length of the pattern, S the number of

sentences, K the number of stored medicine names, L the length of the longest name, V the number of nodes and E the number of edges of the network.

Table 4.1 Time and space complexity of the procedures

Procedure	Time	Extra space	Basis
BUILD-LPS	O(m)	O(m)	Each step advances i or reduces len
KMP-SEARCH	O(n + m)	O(m)	Text index never moves back
DISPLAY-SENTENCES	O(n + S·m)	O(n)	KMP is run on each sentence
EDIT-DISTANCE	O(m·n)	O(m·n)	One table cell per pair of prefixes
CHECK-SPELLING	$O(K \cdot L^2)$	$O(L^2)$	One edit distance per medicine
MAX-FLOW (BFS)	O(V·E) augmentations, $O(V^2)$ per BFS	$O(V^2)$	Edmonds–Karp bound, matrix form

4.5 Worked Examples (Dry Runs)

This section traces each algorithm on a small input. The traces are produced by following exactly the same steps as the Java methods, so that a reader can compare them with the output of the program and confirm that the implementation behaves as the pseudocode describes.

4.5.1 Construction of the LPS array

Table 4.2 traces BUILD-LPS on the pattern ABABCABAB, which was also used in Table 3.1. The variable len holds the length of the current matching prefix, and i is the index being filled. When a mismatch occurs and len is not zero, len falls back to lps[len − 1] and the same i is compared again; this is the step that avoids restarting from the beginning of the pattern.

Table 4.2 Trace of BUILD-LPS for the pattern ABABCABAB

Step	i	len	Comparison	Action
1	1	0	p[1] = B, p[0] = A	mismatch, len=0; lps[1]=0
2	2	0	p[2] = A, p[0] = A	equal; len=1, lps[2]=1
3	3	1	p[3] = B, p[1] = B	equal; len=2, lps[3]=2
4	4	2	p[4] = C, p[2] = A	mismatch, len>0; len=lps[1]=0

Step	i	len	Comparison	Action
5	4	0	p[4] = C, p[0] = A	mismatch, len=0; lps[4]=0
6	5	0	p[5] = A, p[0] = A	equal; len=1, lps[5]=1
7	6	1	p[6] = B, p[1] = B	equal; len=2, lps[6]=2
8	7	2	p[7] = A, p[2] = A	equal; len=3, lps[7]=3
9	8	3	p[8] = B, p[3] = B	equal; len=4, lps[8]=4

The final array is [0, 0, 1, 2, 0, 1, 2, 3, 4]. The last value, 4, shows that the prefix ABAB is also a suffix of the whole pattern. If a full match were found, the search would continue from position 4 of the pattern instead of position 0, which is how overlapping matches are found without re-reading the text.

4.5.2 KMP search

Table 4.3 traces KMP-SEARCH for the pattern ababc in the text abababcabc, whose LPS array is [0, 0, 1, 2, 0]. The text index i never decreases. At step 4 the fourth character matches, but the text has a at position 4 while the pattern expects c, so the pattern index falls back from 4 to lps[3] = 2 without moving i back. The match at position 2 is then completed at step 7.

Table 4.3 Trace of KMP-SEARCH for pattern ababc in text abababcabc

Step	i	j	Text char	Pattern char	Action
1	0	0	a	a	equal: i=1, j=1
2	1	1	b	b	equal: i=2, j=2
3	2	2	a	a	equal: i=3, j=3
4	3	3	b	b	equal: i=4, j=4; mismatch t[4]≠p[4], j=lps[3]=2
5	4	2	a	a	equal: i=5, j=3
6	5	3	b	b	equal: i=6, j=4
7	6	4	c	c	equal: i=7, j=5; full match at 2, j=lps[4]=0
8	7	0	a	a	equal: i=8, j=1
9	8	1	b	b	equal: i=9, j=2; mismatch t[9]≠p[2], j=lps[1]=0
10	9	0	c	a	mismatch with j=0, i=10

The search made 10 comparisons for a text of 10 characters, which agrees with the O(n + m) bound. A naive search would have started again from position 1 after the mismatch at step 4 and compared the characters at positions 1, 2 and 3 a second time.

4.5.3 Edit distance

Table 4.4 shows the complete table dp computed by EDIT-DISTANCE for a = “paracetmol” (rows) and b = “paracetamol” (columns). Row 0 and column 0 hold the distances from the empty string. The diagonal cells for the matching prefix “paracet” stay at 0, because equal characters copy the diagonal value. The first difference appears at the letter m of a against the letter a of b, and the final cell dp[10][11] holds the answer.

Table 4.4 Edit distance table for “paracetmol” and “paracetamol”

	Ø	**p**	**a**	**r**	**a**	**c**	**e**	**t**	**a**	**m**	**o**	**l**
Ø	0	1	2	3	4	5	6	7	8	9	10	11
p	1	0	1	2	3	4	5	6	7	8	9	10
a	2	1	0	1	2	3	4	5	6	7	8	9
r	3	2	1	0	1	2	3	4	5	6	7	8
a	4	3	2	1	0	1	2	3	4	5	6	7
c	5	4	3	2	1	0	1	2	3	4	5	6
e	6	5	4	3	2	1	0	1	2	3	4	5
t	7	6	5	4	3	2	1	0	1	2	3	4
m	8	7	6	5	4	3	2	1	1	1	2	3
o	9	8	7	6	5	4	3	2	2	2	1	2
l	10	9	8	7	6	5	4	3	3	3	2	1

The value in the bottom-right cell is 1: the single operation required is the insertion of the letter a between “t” and “m”. Following the cells back from the bottom-right corner to the top-left corner gives the alignment, in which every character is matched except one inserted letter. Because the distance is between 1 and 3, CHECK-SPELLING reports a possible spelling mistake and suggests “paracetamol”.

4.5.4 Maximum flow

The residual matrices that MAX-FLOW produces for the supply network of Fig. 3.3 are shown below. Table 4.5 is the initial matrix, which is the capacity matrix itself.

Table 4.5 Initial residual matrix

From \ To	**Source**	**Supplier**	**Pharmacy**	**Patient**	**Sink**
Source	0	100	0	0	0
Supplier	0	0	80	20	0

From \ To	Source	Supplier	Pharmacy	Patient	Sink
Pharmacy	0	0	0	60	0
Patient	0	0	0	0	70
Sink	0	0	0	0	0

The first breadth-first search reaches the sink along the path Source → Supplier → Patient → Sink. The bottleneck is the edge Supplier → Patient with capacity 20. After pushing 20 units, the forward capacities on this path fall by 20 and the reverse capacities rise by 20, giving Table 4.6.

Table 4.6 Residual matrix after the first augmentation (20 units)

From \ To	Source	Supplier	Pharmacy	Patient	Sink
Source	0	80	0	0	0
Supplier	20	0	80	0	0
Pharmacy	0	0	0	60	0
Patient	0	20	0	0	50
Sink	0	0	0	20	0

The second search finds the path Source → Supplier → Pharmacy → Patient → Sink. The residual capacity of the last edge is now 50, which is the bottleneck, so 50 units are pushed. Table 4.7 shows the matrix after this step.

Table 4.7 Residual matrix after the second augmentation (50 units)

From \ To	Source	Supplier	Pharmacy	Patient	Sink
Source	0	30	0	0	0
Supplier	70	0	30	0	0
Pharmacy	0	50	0	10	0
Patient	0	20	50	0	0
Sink	0	0	0	70	0

In Table 4.7 the entry from Patient to Sink is 0, so the sink cannot be reached and the algorithm stops. The total flow is 20 + 50 = 70 units. The reverse entries show where the flow actually went: 70 units left the Source, 20 of them went directly from the Supplier to the Patient and 50 went through the Pharmacy.

4.6 Method-level Description

Table 4.8 lists every method of the class PharmaSearchStringMatching with its parameters, return type and purpose, so that the report can be read together with the source code given in Section 4.8.

Table 4.8 Methods of the class PharmaSearchStringMatching

Method	Parameters	Returns	Purpose
buildLPS	String pattern	int[]	Builds the LPS array of the pattern, ignoring case
KMPSearch	String text, String pattern	ArrayList<Integer>	Returns the starting positions of all matches; an empty list for a null or empty pattern
editDistance	String a, String b	int	Computes the Levenshtein distance by dynamic programming
checkSpelling	String query, String[] medicines	void	Finds the closest stored name and prints the distance and the status
bfs	int[][] graph, int source, int sink, int[] parent	boolean	Searches for a path with spare capacity and records the parents
maxFlow	int[][] graph, int source, int sink	int	Repeats augmentation until no path remains and returns the total flow
networkFlow	none	void	Defines the supply network and prints the maximum flow
readCorpus	String filePath	String	Reads the whole file into one string; prints an error on failure
displaySentences	String corpus, String query	void	Prints every sentence that contains the query, numbered
main	String[] args	void	Loads the corpus and runs the menu loop

4.7 Error Handling and Input Validation

The program protects itself against the following abnormal situations.

- **Missing corpus file:** readCorpus catches the IOException, prints “Error reading corpus file.” and returns an empty string. The main method then prints “Corpus file could not be loaded.” together with the expected path and ends the program, so that no search is attempted on missing data.

- **Empty or null pattern:** KMPSearch returns an empty list immediately. Without this check, the expression pattern.charAt(0) would raise an exception when the user presses Enter without typing a query. The menu then reports zero matches and the status NO MATCH FOUND.
- **Invalid menu choice:** because the choice is read as a string and compared with “1”, “2”, “3” and “4”, any other input, including letters and blank lines, falls into the final else branch and produces the message “Invalid choice.” The program therefore cannot fail through a number-format error.
- **Letter case:** both the text and the pattern are converted character by character with Character.toLowerCase, so “ASPIRIN”, “Aspirin” and “aspirin” give the same result.
- **No match:** when the query does not occur, the menu prints the status and no sentence list, and displaySentences prints “No matching sentence found.” if it is ever called with such a query.

The checks are intentionally simple. The program does not remove leading or trailing spaces from the query, so a query typed with an extra space at the end is searched exactly as typed; this behaviour is listed among the limitations in Chapter 6.

4.8 Complete Source Code

The complete source code of the program is reproduced below. The code is in the file PharmaSearchStringMatching.java and the corpus is read from data/pharma_corpus.txt, relative to the folder from which the program is run.

```
import java.io.*;
import java.util.*;

public class PharmaSearchStringMatching {

    // ==================== KMP ====================

    public static int[] buildLPS(String pattern) {
        int[] lps = new int[pattern.length()];
        int len = 0, i = 1;

        while (i < pattern.length()) {
            if (Character.toLowerCase(pattern.charAt(i))
                    == Character.toLowerCase(pattern.charAt(len))) {
                lps[i++] = ++len;
            } else if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i++] = 0;
            }
        }
```

```
        return lps;
    }

    public static ArrayList<Integer> KMPSearch(
            String text, String pattern) {

        ArrayList<Integer> positions = new ArrayList<>();

        if (pattern == null || pattern.isEmpty())
            return positions;

        int[] lps = buildLPS(pattern);
        int i = 0, j = 0;

        while (i < text.length()) {

            if (Character.toLowerCase(text.charAt(i))
                    == Character.toLowerCase(pattern.charAt(j))) {
                i++;
                j++;
            }

            if (j == pattern.length()) {
                positions.add(i - j);
                j = lps[j - 1];
            } else if (i < text.length()
                    && Character.toLowerCase(text.charAt(i))
                    != Character.toLowerCase(pattern.charAt(j))) {

                if (j != 0)
                    j = lps[j - 1];
                else
                    i++;
            }
        }

        return positions;
    }

    // ==================== EDIT DISTANCE ====================

    public static int editDistance(String a, String b) {

        a = a.toLowerCase();
        b = b.toLowerCase();

        int m = a.length();
        int n = b.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++)
            dp[i][0] = i;

        for (int j = 0; j <= n; j++)
            dp[0][j] = j;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {

                if (a.charAt(i - 1) == b.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = 1 + Math.min(
                            dp[i - 1][j],
                            Math.min(
                                    dp[i][j - 1],
                                    dp[i - 1][j - 1]
                            )
                    );
                }
            }
```

```
        }

        return dp[m][n];
    }

    public static void checkSpelling(
            String query, String[] medicines) {

        String closest = "";
        int minDistance = Integer.MAX_VALUE;

        for (String medicine : medicines) {

            int distance = editDistance(query, medicine);

            if (distance < minDistance) {
                minDistance = distance;
                closest = medicine;
            }
        }

        System.out.println("\n===== EDIT DISTANCE =====");
        System.out.println("Entered : " + query);
        System.out.println("Closest : " + closest);
        System.out.println("Distance: " + minDistance);

        if (minDistance == 0)
            System.out.println("Status  : Correct spelling");
        else if (minDistance <= 3)
            System.out.println("Status  : Possible spelling mistake");
        else
            System.out.println("Status  : No close medicine found");
    }

    // ==================== NETWORK FLOW ====================

    public static boolean bfs(
            int[][] graph, int source, int sink, int[] parent) {

        boolean[] visited = new boolean[graph.length];
        Queue<Integer> queue = new LinkedList<>();

        queue.add(source);
        visited[source] = true;
        parent[source] = -1;

        while (!queue.isEmpty()) {

            int u = queue.poll();

            for (int v = 0; v < graph.length; v++) {

                if (!visited[v] && graph[u][v] > 0) {
                    queue.add(v);
                    parent[v] = u;
                    visited[v] = true;
                }
            }
        }

        return visited[sink];
    }

    public static int maxFlow(
            int[][] graph, int source, int sink) {

        int n = graph.length;
        int[][] residual = new int[n][n];

        for (int i = 0; i < n; i++)
            residual[i] = graph[i].clone();
```

```
        int[] parent = new int[n];
        int totalFlow = 0;

        while (bfs(residual, source, sink, parent)) {

            int pathFlow = Integer.MAX_VALUE;

            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                pathFlow = Math.min(
                        pathFlow,
                        residual[u][v]
                );
            }

            for (int v = sink; v != source; v = parent[v]) {

                int u = parent[v];

                residual[u][v] -= pathFlow;
                residual[v][u] += pathFlow;
            }

            totalFlow += pathFlow;
        }

        return totalFlow;
    }

    public static void networkFlow() {

        /*
         * 0 = Source
         * 1 = Supplier
         * 2 = Pharmacy
         * 3 = Patient
         * 4 = Sink
         */

        int[][] graph = {
                {0, 100, 0,  0,  0},
                {0,   0, 80, 20, 0},
                {0,   0,  0, 60, 0},
                {0,   0,  0,  0, 70},
                {0,   0,  0,  0,  0}
        };

        int flow = maxFlow(graph, 0, 4);

        System.out.println("\n===== NETWORK FLOW =====");
        System.out.println("Source -> Supplier -> Pharmacy -> Patient ->
Sink");
        System.out.println("Maximum Medicine Flow: " + flow + " units");
        System.out.println("Algorithm: Ford-Fulkerson");
    }

    // ==================== READ CORPUS ====================

    public static String readCorpus(String filePath) {

        StringBuilder corpus = new StringBuilder();

        try {
            BufferedReader reader =
                    new BufferedReader(new FileReader(filePath));

            String line;

            while ((line = reader.readLine()) != null) {
                corpus.append(line).append("\n");
            }
```

```
            reader.close();

        } catch (IOException e) {
            System.out.println("Error reading corpus file.");
        }

        return corpus.toString();
    }

    // ==================== DISPLAY SENTENCES ====================

    public static void displaySentences(
            String corpus, String query) {

        String[] sentences =
                corpus.split("(?<=[.!?])\\s+");

        int count = 0;

        for (String sentence : sentences) {

            if (!KMPSearch(sentence, query).isEmpty()) {
                count++;
                System.out.println(count + ". " + sentence.trim());
            }
        }

        if (count == 0)
            System.out.println("No matching sentence found.");
    }

    // ==================== MAIN ====================

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        String filePath = "data/pharma_corpus.txt";

        String corpus = readCorpus(filePath);

        if (corpus.isEmpty()) {
            System.out.println("Corpus file could not be loaded.");
            System.out.println("Check: data/pharma_corpus.txt");
            scanner.close();
            return;
        }

        String[] medicines = {
                "paracetamol",
                "amoxicillin",
                "ibuprofen",
                "azithromycin",
                "cetirizine",
                "metformin",
                "omeprazole",
                "aspirin",
                "diclofenac",
                "atorvastatin",
                "insulin",
                "antibiotic",
                "antacid"
        };

        while (true) {

            System.out.println("\n==============================");
            System.out.println("        PHARMA SEARCH");
            System.out.println("==============================");
            System.out.println("1. Search Medicine using KMP");
            System.out.println("2. Check Spelling using Edit Distance");
            System.out.println("3. Calculate Network Flow");
```

```
            System.out.println("4. Exit");
            System.out.println("==============================");

            System.out.print("Enter choice: ");
            String choice = scanner.nextLine();

            // KMP
            if (choice.equals("1")) {

                System.out.print("Enter medicine or keyword: ");
                String query = scanner.nextLine();

                ArrayList<Integer> result =
                        KMPSearch(corpus, query);

                System.out.println("\n===== KMP SEARCH =====");
                System.out.println("Query: " + query);
                System.out.println("Matches Found: " + result.size());

                if (!result.isEmpty()) {
                    System.out.println("Status: MATCH FOUND");
                    System.out.println("\nMatching Sentences:");
                    displaySentences(corpus, query);
                } else {
                    System.out.println("Status: NO MATCH FOUND");
                }
            }

            // Edit Distance
            else if (choice.equals("2")) {

                System.out.print("Enter medicine name: ");
                String query = scanner.nextLine();

                checkSpelling(query, medicines);
            }

            // Network Flow
            else if (choice.equals("3")) {

                networkFlow();
            }

            // Exit
            else if (choice.equals("4")) {

                System.out.println(
                        "Thank you for using Pharma Search!"
                );
                break;
            }

            else {
                System.out.println("Invalid choice.");
            }
        }

        scanner.close();
    }
}
```

4.9 Steps to Compile and Run the Program

The program is a single Java source file and needs no external library. The folder structure and the steps to run it are given below.

Table 4.9 Folder structure of the project

File or folder	Description
PharmaSearchStringMatching.java	Source file containing all methods and the main method
data/	Folder that holds the corpus
data/pharma_corpus.txt	Plain-text pharmaceutical corpus read at start-up

- **Step 1:** install a Java Development Kit (JDK 8 or later) and check the installation with the command java -version.
- **Step 2:** place the source file in a project folder and create the sub-folder data containing the file pharma_corpus.txt.
- **Step 3:** open a terminal in the project folder and compile the program with the command javac PharmaSearchStringMatching.java.
- **Step 4:** run the program from the same folder with the command java PharmaSearchStringMatching. The corpus path is relative, so the program must be started from the project folder, otherwise the message “Corpus file could not be loaded.” is displayed.
- **Step 5:** enter 1, 2 or 3 to use a module, and 4 to exit. Alternatively, the project can be opened in an IDE such as IntelliJ IDEA, Eclipse or VS Code, with the project folder as the working directory.

5. RESULTS

5.1 Experimental Setup

The application was compiled and executed on a standard desktop environment with a JDK installed, as specified in Section 2.3. The corpus file data/pharma_corpus.txt used for testing contained 24 sentences covering eleven medicines and general side-effect and precaution statements, and the spelling checker used the thirteen names listed in Section 4.2. Each module was tested through the menu with typical, boundary and erroneous inputs, and the outputs are reproduced in the figures below. When the program starts, it displays the main menu shown in Fig. 5.1.

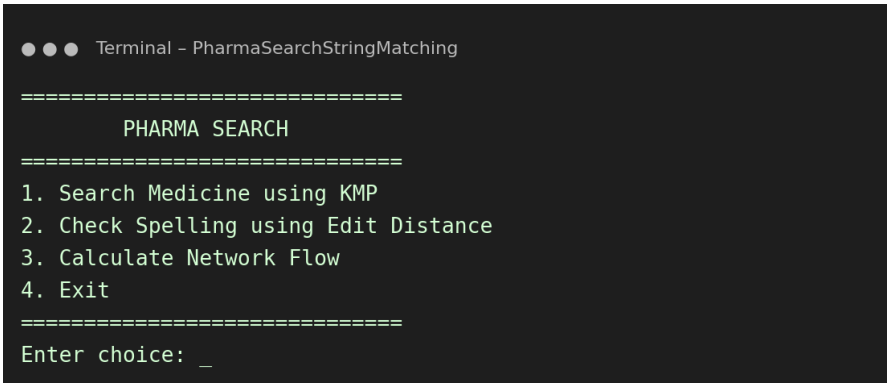


Fig. 5.1 Main Menu of Pharma Search

5.2 KMP Search Results

For the query “paracetamol” the program reported three matches and displayed the three sentences that contain the word, as shown in Fig. 5.2. The first sentence begins with a capital letter (“Paracetamol”) while the query is in lower case, which confirms that the search is case-insensitive.

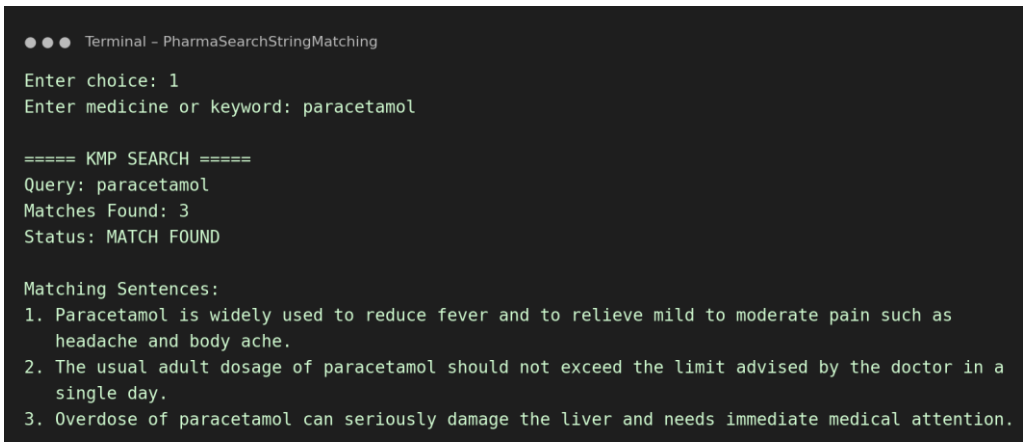


Fig. 5.2 KMP Search – Match Found

When the query does not occur in the corpus, for example “xyz”, the program reports zero matches and the status NO MATCH FOUND without printing any sentence (Fig. 5.3). The query “antibiotic” gave four matches, one of which is inside the word “Antibiotics”, which shows that the search finds the query as a substring and not only as a whole word.

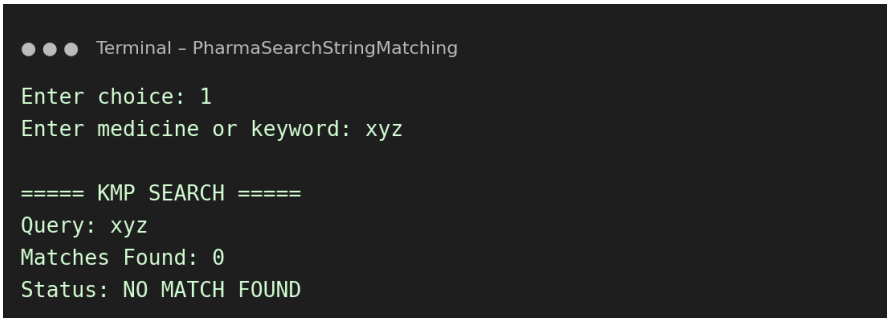


Fig. 5.3 KMP Search – No Match Found

5.3 Edit Distance Results

The misspelt input “paracetmol” was matched with “paracetamol” at an edit distance of 1 and was reported as a possible spelling mistake (Fig. 5.4). The correctly spelt input “amoxicillin” gave a distance of 0 and the status Correct spelling, while the unrelated input “qqqqqqqq” gave a minimum distance of 8 against “aspirin” and the status No close medicine found.

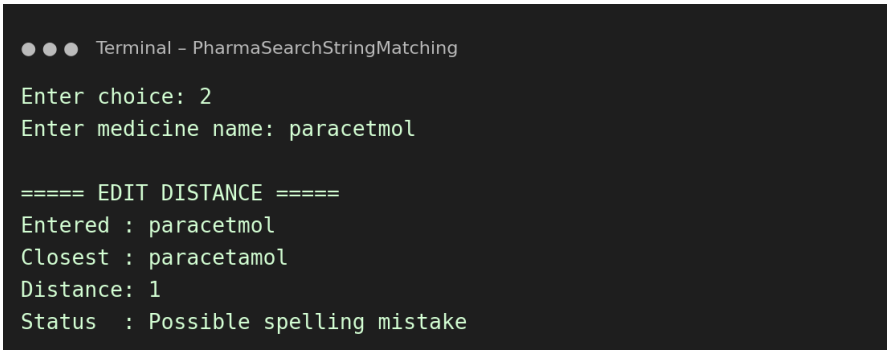


Fig. 5.4 Spelling Check using Edit Distance

5.4 Network Flow Results

For the supply network of Fig. 3.3 the program computed a maximum medicine flow of 70 units (Fig. 5.5). The result can be verified by following the augmenting paths found by the breadth-first search, which are listed in Table 5.1. The first path, Source → Supplier → Patient → Sink, carries 20 units because the edge from the Supplier to the Patient has a capacity of 20. The second path, Source → Supplier → Pharmacy → Patient → Sink, carries the remaining 50 units that the

Patient → Sink edge can accept. After these two augmentations the Patient → Sink edge is saturated and the breadth-first search cannot reach the sink, so the algorithm stops with a total flow of 70.

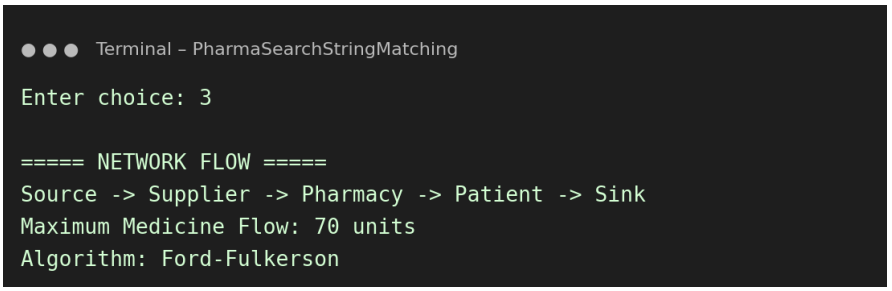


Fig. 5.5 Network Flow Calculation Output

Table 5.1 Augmenting paths found for the supply network

Step	Augmenting path	Bottleneck	Total flow
1	Source → Supplier → Patient → Sink	20	20
2	Source → Supplier → Pharmacy → Patient → Sink	50	70
3	No path from Source to Sink in the residual graph	–	70

The maximum flow equals the capacity of the cut formed by the single edge Patient → Sink (70 units), which is smaller than the 100 units that the source can supply. This agrees with the max-flow min-cut theorem [6] and shows that the last link of the chain, and not the supplier, limits the quantity of medicine reaching the patient.

5.5 Test Cases and Analysis

Table 5.2 summarises the test cases executed in a single run of the program. In every case the observed output matched the expected behaviour of the corresponding requirement of Table 2.1.

Table 5.2 Test cases and observed results

No.	Menu option	Input	Observed output	Result
1	1 (KMP)	paracetamol	3 matches; MATCH FOUND; 3 sentences listed	Pass
2	1 (KMP)	antibiotic	4 matches; MATCH FOUND; 4 sentences listed	Pass
3	1 (KMP)	xyz	0 matches; NO MATCH FOUND	Pass
4	2 (Edit Distance)	paracetmol	Closest: paracetamol; distance 1; possible spelling mistake	Pass

No.	Menu option	Input	Observed output	Result
5	2 (Edit Distance)	amoxicillin	Closest: amoxicillin; distance 0; correct spelling	Pass
6	2 (Edit Distance)	qqqqqqqq	Closest: aspirin; distance 8; no close medicine found	Pass
7	3 (Network Flow)	–	Maximum medicine flow: 70 units	Pass
8	Invalid	9	Invalid choice.	Pass
9	4 (Exit)	–	Thank you for using Pharma Search!	Pass

The results show that the KMP module finds every occurrence regardless of letter case, that the edit distance module separates correct names, near-misses and unrelated words, and that the network flow module identifies the true bottleneck of the supply chain. The menu returned to the main screen after every operation, which confirms that repeated searches work until the user exits.

5.6 Additional KMP Search Tests

Further queries were run to examine how the KMP module behaves on different kinds of input. The results are given in Table 5.3. “Matches” is the number of occurrences found by KMPSearch and “Sentences” is the number of sentences printed by displaySentences.

Table 5.3 Additional KMP search tests

No.	Query	Matches	Sentences	Remark
1	ibuprofen	2	2	Both lines that mention the medicine are listed
2	ASPIRIN	2	2	Upper-case query finds “Aspirin” in the text
3	insulin	2	2	Name appears at the start of two sentences
4	stomach	5	5	Keyword shared by several medicines
5	side effects	1	1	Query containing a space is handled correctly
6	dose	2	2	Matches inside “doses” and “Overdose” (substring search)
7	(empty line)	0	0	Empty pattern returns no positions; no exception

In every test the number of matching sentences equals the number of matches, because no sentence of the test corpus contains the query twice. The two sentences matched for the query “dose” show the consequence of substring

matching. The query is found inside “doses” and inside “Overdose”, which a user searching for the whole word might not expect. This design choice follows from the definition of the KMP algorithm, which finds every occurrence of the pattern as a substring, and it is listed in the limitations. The empty query shows that the guard at the beginning of KMPSearch works: the program does not fail but simply reports no match.

5.7 Additional Spelling Check Tests

Twelve inputs were given to the spelling checker, nine of them misspelt in one of the ways that are common in typing: a missing letter, an extra letter or a wrong letter. The results are in Table 5.4.

Table 5.4 Additional spelling check tests

No.	Entered	Closest	Distance	Status
1	paracetmol	paracetamol	1	Possible mistake
2	ibuprofn	ibuprofen	1	Possible mistake
3	aspirn	aspirin	1	Possible mistake
4	metforman	metformin	1	Possible mistake
5	omeprazol	omeprazole	1	Possible mistake
6	atorvastatn	atorvastatin	1	Possible mistake
7	insulinn	insulin	1	Possible mistake
8	paracetamoll	paracetamol	1	Possible mistake
9	antibiotics	antibiotic	1	Possible mistake
10	azithromycin	azithromycin	0	Correct spelling
11	Cetirizine	cetirizine	0	Correct spelling
12	vitamin	antacid	4	No close medicine found

All nine misspelt names were corrected to the intended medicine at distance 1, and both correctly spelt inputs were reported with distance 0. The comparison is not sensitive to letter case, so “Cetirizine” is accepted. The input “vitamin” is not a stored name; its nearest stored name is “antacid” at distance 4, which is above the threshold of 3, so the program correctly says that no close medicine was found. The result also shows that the closest name for an unrelated word is not meaningful, which is why the status, and not the suggestion alone, must be read.

5.8 Performance Comparison of KMP and Naive Search

To check the practical effect of the linear-time bound, the KMP routine of the project (buildLPS and the search loop, case-insensitive) was compared with a naive search that also ignores case. Both were run on texts of 10 KB to 20,000 KB. Two kinds of text were used. The ordinary text was built by repeating pharmaceutical sentences, and the pattern was “antibiotic”, which does not occur in it. The worst-case text consisted of the letter a repeated, with the pattern “aaaaaaaaab”, which forces a naive search to compare almost the whole pattern at every position. Each time is the mean of five runs after three warm-up runs on the same computer, and is reported in milliseconds in Table 5.5 and in Fig. 5.6.

Table 5.5 Search time of KMP and naive search

Text type	Size (KB)	KMP (ms)	Naive (ms)
Ordinary	10	3.362	1.513
Ordinary	100	1.641	1.533
Ordinary	1,000	7.728	5.117
Ordinary	5,000	42.265	27.884
Ordinary	20,000	151.794	160.034
Worst-case	10	1.536	1.325
Worst-case	100	0.774	5.070
Worst-case	1,000	8.179	50.377
Worst-case	5,000	41.141	270.276
Worst-case	20,000	177.313	1059.926

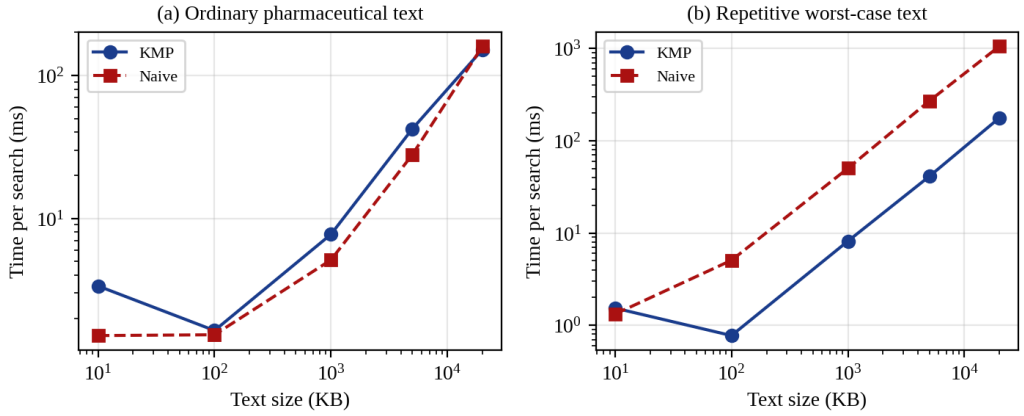


Fig. 5.6 KMP versus Naive Search Time

The results agree with the theory. On the repetitive worst-case text, the naive search is much slower than KMP and the gap widens with the text size: at 20,000 KB it needed about 1,060 ms against about 177 ms for KMP, roughly six times as

long. On ordinary text the two methods took similar times, and for the largest text the difference was small. This is expected, because in ordinary text a mismatch usually occurs at the first or second character of the pattern, so the naive method rarely repeats work. The slightly higher KMP time at 10 KB is within the noise of the Java just-in-time compiler for such short runs. These measurements were taken on one computer, with a limited number of repetitions, and are meant to show the trend and not exact figures.

The comparison shows why KMP is preferred when the guarantee matters. For ordinary text it costs no more than the naive method, and for text with long partial matches it protects the program from a large slowdown. Because the application is interactive, a bounded response time for every query is more valuable than a faster response for typical queries only.

5.9 Discussion of Results against the Objectives

Table 5.6 relates the results of testing to the objectives listed in Section 1.4.

Table 5.6 Achievement of the objectives

Objective	Evidence from testing
KMP search with the LPS array	Correct matches for all queries of Tables 5.2 and 5.3; trace in Table 4.3 agrees with the program
Count and display of matching sentences	Count and numbered sentences shown for every matching query (Figs. 5.2 and 5.3)
Edit distance for spelling errors	Nine misspelt names corrected at distance 1; unrelated word rejected (Table 5.4)
Maximum flow of the supply network	70 units, verified by the augmenting paths and by the minimum cut (Section 5.4)
Simple menu with repeated operations	Menu returned after every operation; invalid input handled (Table 5.2)
Practical use of the Java concepts	Arrays, loops, methods, collections, file handling and input processing used in a single program

All the objectives were met. The main finding of the performance study is that the benefit of KMP is guaranteed worst-case behaviour, and not a speed-up on every input. The main finding of the functional tests is that the three modules complement each other: a user who enters a misspelt name receives a suggestion from the edit distance module, and can then search the corrected name with the KMP module.

5.10 Scalability and Memory Considerations

The memory used by Pharma Search is dominated by the corpus string and, during a spelling check, by the dp table. The corpus is held as a single String, so its size is proportional to the number of characters in the file. The LPS array needs one integer per character of the query, which is negligible for names and keywords. The dp table needs (m + 1) × (n + 1) integers for two names of lengths m and n; for medicine names of fewer than 20 letters it holds fewer than 500 integers. The residual matrix of the flow module needs V × V integers, which is 25 for the five nodes of the supply network. Table 5.7 summarises the growth of time and memory of each module.

Table 5.7 Growth of time and memory with the input size

Module	Time grows with	Memory grows with	Practical limit
KMP search	Corpus length n and query length m	n (corpus) + m (LPS)	Available heap for the corpus string
Sentence display	n plus number of sentences × m	Array of sentences, about n	Same as the corpus
Edit distance	m × n for each stored name	m × n	Very small for names
Network flow	Number of augmentations × V^2	V^2	Matrix size for large networks

For a corpus of several megabytes, which already holds many thousand pages of text, the search remains linear and finishes within a fraction of a second, as the measurements of Section 5.8 show. When the corpus grows beyond the memory of the computer, the file would have to be searched in blocks or indexed, which is listed under future work. The sentence display step splits the entire corpus on every query, which is acceptable for the present size, but for a larger corpus the sentence boundaries should be computed once and reused.

6. CHALLENGES & LIMITATIONS

6.1 Issues faced during development

- **Building the LPS array correctly:** the fall-back step len ← lps[len − 1] in BUILD-LPS and the corresponding reset of j during the search are easy to get wrong and, if wrong, either skip genuine matches or loop forever.
- **Case-insensitive comparison:** the corpus contains capitalised names at the start of sentences, while users type in lower case, so a plain character comparison missed valid matches.
- **Showing context for a match:** a position in the corpus is not meaningful to a user, so the matching sentences had to be extracted from the text.
- **Choosing the spelling threshold:** a very small threshold rejects genuine typing errors in long names, whereas a large one accepts unrelated words as suggestions.
- **Modelling the supply chain:** representing the supplier, pharmacy and patient as a flow network and understanding why reverse edges must be maintained in the residual graph was conceptually difficult.
- **Missing or unreadable corpus file:** the program must start in a clear state even when the file is not present in the expected folder.

6.2 Solutions applied

- The LPS construction and the search loop were traced by hand on small patterns, such as the one in Table 3.1, and the results were compared with the program output until they agreed.
- Every comparison applies Character.toLowerCase to both characters, which gives a case-insensitive search without making a lower-case copy of the corpus.
- The corpus is split into sentences with the regular expression (?<=[.!?])\s+ and KMP is applied to each sentence to list those that contain the query.
- A threshold of 3 edits was adopted after trying typical misspellings, so that single-letter errors such as “paracetmol” are caught while unrelated words are not suggested.

- The flow network was drawn on paper first, and the residual matrix was updated with both forward and reverse changes in MAX-FLOW; the result was checked against the minimum cut.
- readCorpus catches the IOException and main stops with a message that tells the user which file to check.

6.3 Limitations of the System

- The application is console-based and has no graphical interface.
- The search is a substring search, so a query such as “antibiotic” also matches “antibiotics”; whole-word search is not supported.
- The sentence splitter assumes that every full stop ends a sentence, so abbreviations such as “Dr.” can split a sentence in the wrong place.
- The spelling checker compares the input with only thirteen stored names and reports only the single closest one; ties are resolved in favour of the name that appears first in the list.
- The whole corpus is held in memory as one string and is scanned afresh on every search, because no index is built.
- The supply network is fixed in the code and cannot be entered by the user.
- The corpus is static and has not been verified clinically; the output is for demonstration and must not be used as medical advice.

6.4 Safety and Ethical Considerations

Pharmaceutical information can influence decisions about health, so the use of any search tool in this domain needs care. Pharma Search returns sentences from a text file exactly as they are written in that file, and the accuracy of the output is therefore only as good as the accuracy of the corpus. The corpus used in the project is a sample prepared for demonstration. If the application were used with real data, the corpus would have to be taken from verified sources such as approved drug leaflets, and it would have to be reviewed and updated regularly.

The spelling suggestion must also be treated as a suggestion and not as a confirmation. Many medicine names differ from each other by only one or two letters, and an automatic correction to the wrong medicine could mislead a user. For this reason, the program displays the entered name, the closest name and the

distance together, and leaves the decision to the user. The application does not store any personal data, and it does not give dosage advice, diagnosis or prescriptions. A notice that the output is for educational purposes only should accompany any future version that is made available to the public.

6.5 Lessons Learnt from Testing

Testing showed that most errors in algorithm code are caused by boundary conditions and not by the main logic. The empty pattern, a pattern longer than the text, a query that matches at the very end of the corpus and a query that matches several times in one sentence are examples. Tracing the algorithms by hand on small inputs, as done in Section 4.5, was the most effective way to find such errors, because it revealed the state of every variable at every step. The performance measurements also showed that an algorithm with a better worst-case bound may not be faster on typical data, and that conclusions about speed must be supported by measurements on realistic inputs.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

Pharma Search demonstrates how algorithms from the Data Structures and Algorithms-3 course can be combined in one practical application. The KMP module searches a pharmaceutical corpus in linear time, ignores letter case, reports the number of matches and displays the sentences in which the term occurs. The edit distance module detects probable spelling mistakes in medicine names, and the network flow module computes the maximum quantity of medicine that can move from the supplier to the patient. All three modules were tested through the menu, and the observed outputs agreed with the expected behaviour, including the maximum flow of 70 units verified by the minimum cut.

The project also gave practical experience of file handling, arrays, collections, loops, methods and user input processing in Java. Its main limitations are the fixed medicine list, the substring-based matching and the console interface, and these provide clear directions for further work.

7.2 Future Work

- Provide a graphical or web interface with highlighting of the matched keyword inside each sentence, and display the positions of the matches along with their count.
- Support whole-word search, searching for several medicines at once with the Aho–Corasick algorithm, and approximate matching inside the corpus using edit distance.
- Suggest the top few closest medicine names instead of only one, and build the list of names automatically from the corpus.
- Build an index, such as an inverted index or a suffix array, to make repeated searches on very large corpora faster.
- Allow the user or a database to define the supply network and its capacities so that real pharmacy stock data can be used.
- Use larger, verified pharmaceutical sources for the corpus and add support for regional languages.

7.3 Skills learned during development

- Understanding and implementing the KMP algorithm and the LPS array, and analysing the time complexity of string matching.
- Applying dynamic programming to the edit distance problem.
- Modelling a real-world problem as a flow network and solving it with the Ford–Fulkerson method and breadth-first search.
- Java programming skills: file handling with BufferedReader, collections, regular expressions, modular design with static methods and console input handling.
- Testing, debugging and tracing algorithms by hand, and writing pseudocode and technical documentation.
- Working as a team on design, coding, testing and report writing.

7.4 Roadmap of Future Enhancements

Table 7.1 arranges the enhancements suggested in Section 7.2 in a suggested order of implementation, with the algorithm or technique that would be used for each.

Table 7.1 Roadmap of future enhancements

No.	Enhancement	Technique	Expected benefit
1	Whole-word and highlighted search	Boundary check after a KMP match	More precise results
2	Top-k spelling suggestions	Edit distance with sorting	Better help for typing errors
3	Display of match positions	Existing list of positions	Easier location in the text
4	Multi-keyword search	Aho–Corasick automaton	Several medicines in one pass
5	Approximate search inside the corpus	Edit distance on windows	Finds misspelt words in the text
6	User-defined supply network	Input of capacities from a file	Real stock planning
7	Graphical or web interface	Java Swing or web framework	Easier use
8	Indexed corpus	Inverted index or suffix array	Fast search on large corpora

7.5 Concluding Remarks

Pharma Search began as a course project on algorithms, but it brings out a general lesson. The same small set of ideas, scanning a text without going back, filling a table of sub-problems and pushing flow along augmenting paths, can be applied to three quite different questions in one domain: where a word occurs, how close two words are, and how much of a commodity can be delivered. Understanding the algorithms well enough to implement, trace and measure them was the real outcome of the project, and the application is the visible result of that understanding.

REFERENCES

[1] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, “Fast pattern matching in strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[2] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[3] R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” Journal of the ACM, vol. 21, no. 1, pp. 168–173, 1974.

[4] L. R. Ford Jr. and D. R. Fulkerson, “Maximal flow through a network,” Canadian Journal of Mathematics, vol. 8, pp. 399–404, 1956.

[5] J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” Journal of the ACM, vol. 19, no. 2, pp. 248–264, 1972.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[7] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.

[8] Oracle, “Java Platform, Standard Edition Documentation.” [Online]. Available: https://docs.oracle.com/en/java/javase/

[9] R. S. Boyer and J. S. Moore, “A fast string searching algorithm,” Communications of the ACM, vol. 20, no. 10, pp. 762–772, 1977.

[10] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.

```java
import java.util.*;

public class PharmaSearchStringMatching {

    static Scanner sc = new Scanner(System.in);

    // Medicine database for this educational project
    static class Medicine {
        String name, category, uses, problems;
        String dosage, sideEffects, precautions, alternatives;
        int stock;

        Medicine(String name, String category, String uses,
                 String problems, String dosage, String sideEffects,
                 String precautions, String alternatives, int stock) {
            this.name = name;
            this.category = category;
            this.uses = uses;
```

```
PharmaSearchProject > J PharmaSearchStringMatching.java > ...
 4  public class PharmaSearchStringMatching {
 9      static class Medicine {
14          Medicine(String name, String category, String uses,
21              this.dosage = dosage;
22              this.sideEffects = sideEffects;
23              this.precautions = precautions;
24              this.alternatives = alternatives;
25              this.stock = stock;
26          }
27      }
28
29      static Medicine[] medicines = {
30          new Medicine(name: "Paracetamol", category: "Pain reliever and fever reducer",
31              uses: "Helps relieve mild to moderate pain and reduce fever.",
32              problems: "Fever, headache and body aches.",
33              dosage: "Follow the product label or professional advice.",
34              sideEffects: "Overdose can cause serious liver damage.",
35              precautions: "Avoid taking multiple products containing paracetamol.",
36              "Ibuprofen may help with some types of pain or fever, "
37              + "but it is not suitable for everyone. Ask a pharmacist."
```

```java
public class PharmaSearchStringMatching {
    static Medicine[] medicines = {

        new Medicine(name: "Ibuprofen", category: "NSAID pain reliever",
            uses: "Helps relieve pain, inflammation and fever.",
            problems: "Some headaches, muscle pain and inflammatory conditions.",
            dosage: "Use only as directed on the label or by a professional.",
            "May cause stomach upset; serious stomach or kidney "
            + "problems can occur.",
            "May be unsuitable with stomach ulcers, kidney disease, "
            + "some heart conditions or during certain stages of pregnancy.",
            "Paracetamol may be an option for some pain or fever. "
            + "Ask a pharmacist which option is appropriate.",
            stock: 80),

        new Medicine(name: "Amoxicillin", category: "Penicillin antibiotic",
            uses: "Treats certain bacterial infections when prescribed.",
            problems: "Some bacterial ear, throat, chest or other infections.",
            dosage: "Take only as prescribed; complete the prescribed course.",
            sideEffects: "Nausea, diarrhoea or allergic reactions.",
```

```
PharmaSearchProject > J PharmaSearchStringMatching.java > ...
 4  public class PharmaSearchStringMatching {
29      static Medicine[] medicines = {
71
72          new Medicine(name: "Cetirizine", category: "Antihistamine",
73              uses: "Helps relieve allergy symptoms.",
74              problems: "Sneezing, runny nose, itching and watery eyes.",
75              dosage: "Follow the label or professional advice.",
76              sideEffects: "May cause drowsiness or dry mouth.",
77              "Use caution when driving if drowsy; check suitability "
78              + "for children, pregnancy or kidney disease.",
79              "Loratadine is another antihistamine that may be suitable "
80              + "for some people. Ask a pharmacist.",
81              stock: 60),
82
83          new Medicine(name: "Metformin", category: "Antidiabetic medicine",
84              uses: "Helps control blood glucose in type 2 diabetes.",
85              problems: "Management of type 2 diabetes.",
86              dosage: "Use only as prescribed.",
87              sideEffects: "Nausea, diarrhoea and stomach discomfort.",
88              "Kidney function and other medical conditions need "
```

```
PharmaSearchStringMatching.java ×  PharmaSearch.class  PharmaSearchStringMatching$Medicine.class  Ph
PharmaSearchProject > J PharmaSearchStringMatching.java > ...
 4  public class PharmaSearchStringMatching {
29      static Medicine[] medicines = {
116         new Medicine(name: "Diclofenac", category: "NSAID pain reliever",
117             uses: "Helps relieve pain and inflammation.",
118             problems: "Some joint, muscle and inflammatory pain.",
119             dosage: "Use only as directed by a healthcare professional.",
120             sideEffects: "Stomach upset; cardiovascular, kidney or bleeding risks.",
121             "Discuss heart disease, ulcers, kidney disease and "
122             + "other medicines with a professional.",
123             "Paracetamol may suit some pain conditions, but the choice "
124             + "depends on the person and the reason for treatment.",
125             stock: 25),
126
127         new Medicine(name: "Atorvastatin", category: "Cholesterol-lowering statin",
128             uses: "Lowers LDL cholesterol and helps reduce cardiovascular risk.",
129             problems: "High cholesterol and prevention of certain cardiovascular events.",
130             dosage: "Take only as prescribed.",
131             sideEffects: "Muscle symptoms or digestive discomfort may occur.",
132             precautions: "Report unexplained muscle pain and check medicine interactions.",
```